- Non-UI tests can also be used to verify nonfunctional requirements, such as performance.
- Implementing non-UI acceptance tests, and the corresponding application code, is also a great way to discover what components your application needs and to design clean, effective APIs within your application.

Let's start with how you should find the correct balance between UI and non-UI acceptance tests.

9.1 Balancing UI and non-UI acceptance tests

The ideal balance between UI and non-UI tests will naturally vary from project to project, depending on the nature of the application being built, the features being developed, and the technologies used. For some requirements, web or UI tests will be a natural fit, but for others, non-UI-based testing is more appropriate. Still other tests may need a mixture of both approaches.

Some web applications choose to keep business logic on the client side to a minimum, with the bulk of the business logic being performed on the server side in a well-defined service layer. This can be a deliberate design decision or a consequence of the chosen technology stack. This approach is illustrated in figure 9.2.

In this sort of application, automated web tests will typically be used to illustrate and verify the user's journey through the application, to ensure that data is submitted to the server correctly, to check that form validation messages are displayed correctly, to see that results from the server are rendered accurately, and so on. The web tests also

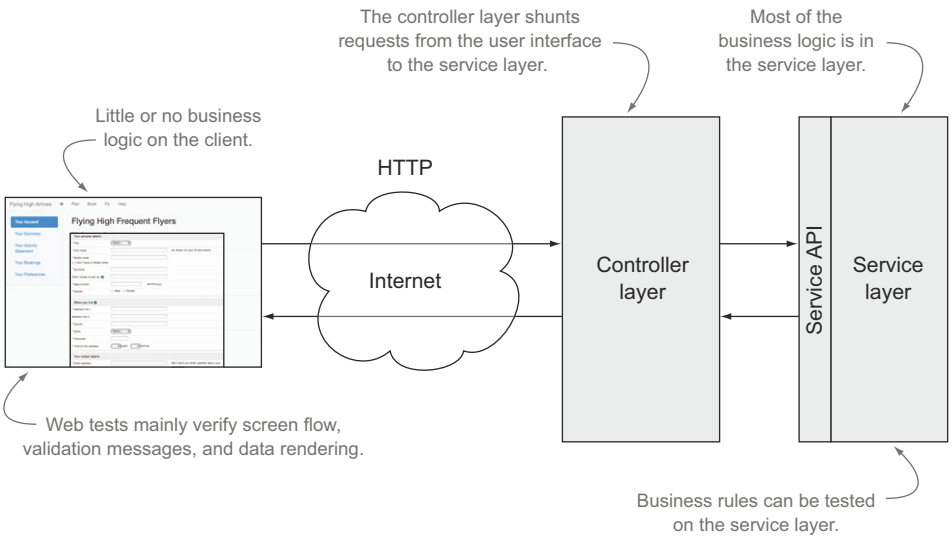


Figure 9.2 When an application has little business logic in the UI, much of the business logic can be tested directly against the service layer.